DR. GANESH ACADEMY 学
drganeshacademy@gmail.com

DSA + LLD + HLD + System Design Program in Tamil

From Problem Solving to Architecture Thinking

Course Overview

This program is designed to take you from **coding fundamentals** to **system architecture mastery** one structured topic at a time. It's not just about solving algorithmic problems; it's about **thinking like an engineer** who can design, scale, and optimize entire systems used by millions.

You'll journey through the three critical layers of software development:

1. **DSA (Data Structures & Algorithms)** – for problem-solving and optimization.
2. **LLD (Low-Level Design)** – for writing clean, modular, object-oriented code.
3. **HLD (High-Level Design)** – for building scalable, distributed systems like Uber, Netflix, or Twitter.

Why This Program?

Most developers stop after learning syntax and basic problem solving. This course bridges the *next leap,* from writing correct code to **designing production-grade systems**.

How It Helps

You'll learn **how software really works behind the scenes**,
how to make **trade-offs** between speed, scalability, and reliability,
and how to **structure your solutions** for both *interview success* and *real-world impact*.

Who Is This For?

This program is built for anyone serious about levelling up their technical foundation,

Students & Freshers Build strong DSA fundamentals and crack product company interviews.

Working Developers Strengthen system thinking, code architecture, and design skills.

Aspiring Backend Engineers Master distributed systems and scalable design concepts.

Total Fee: 20,000 Rs. (Can be paid in installments)

No advanced prerequisites — just basic coding knowledge and curiosity to learn deeply.

Phase 1 – DSA Foundations (Days 1–30)

Focus: Core problem-solving, time-space trade-offs, and algorithmic mastery.

Topic	Description	Detailed Concepts / Algorithms
1. Introduction to DSA	Understand what data structures are and how algorithms operate on them.	Big-O Notation, Time & Space Complexity, Asymptotic Analysis
2. Arrays & Strings	Backbone of most problems — focus on manipulation and traversal.	Prefix Sum, Sliding Window, Two Pointers, Kadane's Algorithm
3. Linked Lists	Dynamic memory management and pointer manipulation.	Single, Double, Circular, Floyd's Cycle, Reversal
4. Stacks & Queues	LIFO/FIFO concepts critical in recursion and scheduling.	Stack via Array/LL, Queue, Deque, Monotonic Stack, Expression Evaluation
5. Recursion & Backtracking	Foundation of tree traversal and combinatorial problems.	N-Queens, Sudoku Solver, Subset Generation, Permutations
6. Searching Algorithms	Efficiently find elements in sorted data.	Linear Search, Binary Search, Ternary Search, Binary Search on Answer
7. Sorting Algorithms	Understanding stable and in-place sorting techniques.	Merge Sort, Quick Sort, Heap Sort, Counting Sort
8. Hashing & Hashmaps	Constant-time lookup and efficient mapping.	Collision Handling, Hash Functions, LRU Cache
9. Trees	Hierarchical data representation used in databases, OS, etc.	Traversals, BST, Heap, Trie, Segment Tree
10. Graphs	Model real-world networks and dependencies.	BFS, DFS, Dijkstra, Bellman-Ford, Kruskal, Prim, Topo Sort
11. Dynamic Programming	Optimize recursive solutions using overlapping subproblems.	Fibonacci, LIS, Knapsack, Matrix Chain Multiplication
12. Greedy Algorithms	Local optimal choices leading to global solutions.	Activity Selection, Huffman Encoding, Kruskal's MST
13. Bit Manipulation	Efficient computation and optimization tricks.	XOR patterns, Subset Generation, Masking
14. Divide & Conquer	Breaking problems into subproblems.	Merge Sort, Binary Search, Karatsuba Multiplication
15. Practice Problems & Mock Interviews	Apply concepts on LeetCode/HackerRank /Codeforces.	Mixed-level real-world DSA problems

Phase 2 – Low-Level Design (LLD) (Days 31–60)

Focus: Object-Oriented Design, Design Patterns, and Class Modeling.

Topic	Description	Detailed Concepts / Patterns
1. OOP Principles	Encapsulation, Inheritance, Abstraction, Polymorphism	UML Diagrams, Interfaces, Composition vs Inheritance
2. SOLID Principles	Design clean and scalable classes.	SRP, OCP, LSP, ISP, DIP
3. Design Patterns - Creational	Object creation techniques.	Singleton, Factory, Builder, Prototype
4. Design Patterns - Structural	Object composition and relationship.	Adapter, Decorator, Facade, Composite, Proxy
5. Design Patterns - Behavioral	Communication between objects.	Strategy, Observer, Command, State, Template
6. Case Studies in LLD	Model real systems.	Parking Lot, Tic-Tac-Toe, Elevator, Movie Ticket Booking
7. Design Principles Recap	Review of reusability, modularity, and extensibility.	Class diagram reviews and refactoring examples

Phase 3 – High-Level Design (HLD) (Days 61–90)

Focus: System architecture, scalability, reliability, and distributed systems.

Topic	Description	Detailed Systems / Algorithms
1. Introduction to System Design	Learn what system design is and why scalability matters.	Functional vs Non-functional Requirements, Load Metrics
2. Scalability Fundamentals	Understand scaling techniques.	Vertical & Horizontal Scaling, Sharding, Partitioning
3. Load Balancing	Distribute load efficiently across servers.	Round Robin, Consistent Hashing, Sticky Sessions
4. Caching	Improve performance using in-memory stores.	Redis, Memcached, Write-through vs Write-back
5. Database Design	Choose and design data stores.	SQL vs NoSQL, ER Modeling, Indexing, Normalization
6. Messaging & Queues	Asynchronous communication in systems.	RabbitMQ, Kafka, Pub/Sub, Event-driven Architecture
7. API Design & Rate Limiting	Building scalable APIs.	REST, gRPC, Rate Limiting, Throttling, Pagination
8. Distributed System Concepts	Consistency, Availability, and Partition Tolerance.	CAP Theorem, Consensus (Paxos, Raft)
9. Monitoring & Logging	Observability and health tracking.	Prometheus, Grafana, ELK Stack
10. Designing Real Systems	End-to-end case studies.	URL Shortener, WhatsApp, Twitter Feed, Uber, YouTube, Netflix
11. Security & Fault Tolerance	Keep systems safe and reliable.	Authentication, Authorization, Rate Limiting, Circuit Breakers
12. System Design Mock Interviews	Apply knowledge with peers.	Whiteboard design and trade-off discussions

🧱 Final Projects & Evaluation

- Project 1: Design a Scalable Notification System (using LLD + HLD concepts)
- Project 2: Implement a Custom Cache System (LRU, LFU)
- Project 3: Build a Mini Twitter (feed ranking, follower system, message queue)
- Project 4: Simulate an Elevator System (LLD Focus)

⏱️ Duration

- **90 Days Structured Learning Plan**
- **1 Topic per Day (with tasks & problem-solving challenges)**

By the end of this journey, you won't just be solving problems — you'll be **designing solutions**.

You'll think about **latency, fault tolerance, throughput, and availability** — the real engineering metrics that matter.
You'll transition from a **coder** to a **complete engineer** who can think, design, and build systems that scale.
This course is built to help learners like you connect the dots, from algorithmic logic to architectural thinking.
Whether you're preparing for interviews or aiming to level up in your career, this roadmap will make you confident not just in coding but in designing real-world systems